김도윤 · 이력서 / PROFILE

멀티 에이전트 오케스트레이션 · LangGraph · FastAPI | AI Agent Engineer

지원 요약

신입 AI 에이전트 엔지니어 지원자. Python 기반 LLM 애플리케이션을 직접 설계·구현하고, 도구 호출·상태 관리·오류 복구를 결합한 업무형 에이전트에 관심이 있습니다. 데모 구현에서 끝내지 않고 실패 시나리오와 측정 지표를 함께 정의하는 것을 강점으로 삼습니다.

기본 정보

이메일: doyun.kim@example.com | 거주: 서울(가상) | 지원 분야: AI Agent / LLM Application
학력: 한빛대학교 컴퓨터공학과 학사 (2026.02 졸업, 가상)
경험: 2025.07~2025.12 교내 AI 서비스 연구팀 학부연구생, 2026.03~2026.08 에이전트 개발 프로젝트

핵심 역량

- LangGraph의 StateGraph, 조건부 분기, 도구 호출 및 재시도 구조 구현
- FastAPI 기반 REST API, WebSocket 스트리밍, Pydantic 스키마 검증
- PostgreSQL/Redis 기반 작업 상태와 실행 로그 관리, Docker Compose 개발 환경
- LangSmith 스타일의 실행 추적 및 데이터셋 기반 에이전트 품질 평가

프로젝트 한눈에 보기

프로젝트 A | 고객지원 업무 처리 AI 에이전트 (2026.03~08)

FAQ 검색→주문 상태 조회→정책 검토→응답 초안 작성까지 이어지는 그래프형 에이전트. 3인 팀에서 백엔드·에이전트 흐름 설계 담당.

프로젝트 B | 서버 운영 점검 Copilot (2025.09~12)

운영 로그와 메트릭을 검색해 점검 항목을 제안하는 읽기 전용 내부 도구. 2인 팀에서 로그 수집 및 툴 권한 설계 담당.

목표 직무

에이전트의 의사결정을 실제 업무 데이터와 연결하고, 도구 실행의 안전성·관측성·평가 체계를 갖춘 서비스를 개발하고 싶습니다.

김도윤 · 대표 프로젝트 / ARCHITECTURE

멀티 에이전트 오케스트레이션 · LangGraph · FastAPI | AI Agent Engineer

프로젝트 배경과 목표

고객지원 담당자가 주문 조회, 반품 규정 확인, 답변 작성에 사용하는 화면이 분산되어 있어 처리 시간이 길어지는 상황을 가정했습니다. 목표는 FAQ만 답하는 챗봇이 아니라 상황에 맞는 도구를 호출하고, 결론에 근거와 불확실성을 표시하는 업무 보조 에이전트를 만드는 것이었습니다.

아키텍처

> 입력 정규화 노드 → 요청 분류 노드 → 지식 검색/주문 조회 도구 노드 → 정책 확인 노드 → 응답 생성 노드 → 검증 노드
>
> LangGraph TypedDict 상태에 request_id, intent, retrieved_doc_ids, tool_results, safety_flags, response_draft를 보관
>
> 주문번호가 없거나 권한이 부족한 경우 조건부 Edge로 추가 정보 요청 노드에 분기
>
> 쓰기 작업(반품 접수)은 실제 실행 대신 사용자 확인을 요구하도록 설계

담당 업무

1. 그래프 및 도구 계약 설계

도구 입력을 Pydantic 모델로 정의하고 주문 조회 응답의 필수 필드 및 오류 코드를 고정했습니다. 모델이 잘못된 인수를 만들면 도구 실행을 차단하고 인수 수정 경로로 이동하도록 구성했습니다.

2. 대화 상태 관리

대화 ID별 요청 컨텍스트는 Redis에 단기 저장하고, 최종 처리 결과는 PostgreSQL에 저장했습니다. 긴 대화는 요약하고 민감정보를 상태에 중복 보관하지 않았습니다.

3. 모니터링

노드별 지연·도구 호출 수·실패 유형을 기록하고, 실패 단계가 구분되도록 tracing 필드를 추가했습니다.

기술 스택

Python 3.12, FastAPI, LangGraph, Pydantic, PostgreSQL, Redis, Docker, pytest, React(간단한 운영 화면)

김도윤 · 문제 해결 / TECHNICAL DEPTH

멀티 에이전트 오케스트레이션 · LangGraph · FastAPI | AI Agent Engineer

핵심 문제 1 | 잘못된 도구 선택

초기 프로토타입에서는 배송 문의를 반품 정책 검색으로 잘못 분류하거나, 주문번호 없이 조회 도구를 호출하는 문제가 발생했습니다.

- 요청 유형별 필수 슬롯을 명시하고, 분류 결과에 confidence를 포함해 불확실한 요청은 확인 질문으로 전환
- 도구 호출 전 파라미터 검증 및 권한 확인을 추가하고, 도구 오류를 사용자에게 그대로 노출하지 않도록 응답 표준화
- 동일한 질문에서 재시도가 무한 반복되지 않도록 호출 횟수 상한 설정

핵심 문제 2 | 답변의 근거 부족

검색된 FAQ 문서와 주문 조회 결과가 서로 다른 경우, LLM이 한쪽 사실을 선택해 단정하는 문제가 있었습니다.

- 정책 문서의 유효일과 주문 데이터의 실제 상태를 구분하고 충돌 시 불확실성을 표시
- 답변 문장마다 source_document_id 또는 tool_result_id를 연결하는 검증 규칙 작성
- 고객에게 노출하지 않는 내부 검증 필드에 실제 근거 구간을 저장

검증 방법과 결과 (가상 프로젝트 측정값)

팀에서 구성한 120개 문의 시나리오를 동일 조건으로 반복 평가했습니다. 적절한 도구 경로 선택 비율은 78%→91%, 필수 정보가 없는 상태에서의 부적절한 조회 호출은 19건→3건으로 감소했습니다. 다만 복합 문의에서는 추가 확인 질문 비율이 증가해 후속 개선 과제로 남겼습니다.

후속 개선 과제

- 정책 변경 이력의 유효 시점 검색
- 동일 고객 문의의 재진입 및 중단 복구 처리
- 에이전트 비용·지연·성공률을 함께 반영하는 평가 기준 수립

김도윤 · 협업 및 추가 경험 / EXPERIENCE

멀티 에이전트 오케스트레이션 · LangGraph · FastAPI | AI Agent Engineer

프로젝트 B | 서버 운영 점검 Copilot

교내 실습 서버의 PostgreSQL 로그와 시스템 메트릭을 읽기 전용으로 조회하는 점검 도구를 제작했습니다. 자연어 질문을 곧바로 쉘 명령으로 바꾸지 않고, 사전에 정의한 도구 목록에서만 호출하도록 제한했습니다.

담당 구현

- CPU/메모리 사용률, 디스크 여유 공간, 느린 SQL 로그를 조회하는 도구 구현
- 운영 점검 질의를 템플릿화하고 read-only DB 계정으로 실행 권한 제한
- 최근 15분/1시간/24시간을 명시적으로 선택하게 하여 시간 범위 혼동 방지
- 로그가 부족할 때 장애 원인을 단정하지 않고 추가 확인 항목 제시

실패와 개선

문제: 관측 지표가 서로 다른 시각을 가리킴

서버 로컬 시간과 로그 수집 시각이 달라 동일 장애를 다른 사건으로 묶는 문제가 있었습니다. 모든 기록을 UTC 기반 이벤트 시각으로 정규화하고 ingestion_time을 별도 보관했습니다.

문제: 과도한 로그 본문 전송

토큰 사용량과 응답 지연을 줄이기 위해 최근 시간 범위 필터·오류 레벨 필터와 요약을 선행하도록 변경했습니다.

협업 경험

팀원은 프론트엔드 화면을, 저는 API/도구 계약을 담당했습니다. 응답 필드가 자주 바뀌어 통합 시 오류가 발생하자 OpenAPI 스펙을 고정하고 샘플 JSON과 계약 테스트를 추가했습니다. 이 경험을 통해 도구 인터페이스를 먼저 합의하는 습관을 갖게 되었습니다.

김도윤 · 기술 및 성장 계획 / SKILLS

멀티 에이전트 오케스트레이션 · LangGraph · FastAPI | AI Agent Engineer

기술 역량 세부

Python: 비동기 함수, 타입 힌트, pytest fixture, 예외 처리

LLM: 프롬프트 분리, tool calling, structured output, 실패 재시도

Backend: FastAPI, WebSocket, JWT 기본 이해, Pydantic

Data: PostgreSQL, Redis, SQL, 데이터 모델링 기초

Ops: Docker Compose, GitHub Actions 기초, 로그 및 지표 기반 디버깅

학습 및 활동

- 2025 대학 AI 서비스 해커톤: 팀 프로젝트 백엔드 담당, 프로토타입 시연
- 2026 에이전트 워크플로 세미나: 조건부 Edge와 복구 설계를 주제로 팀 내부 발표
- 주간 개발 기록에 가설·실험 결과·실패 원인을 정리하며 코드 리뷰 참여

입사 후 기여 계획

첫 1개월은 기존 도구·권한·데이터 흐름을 파악하고 테스트 케이스를 보강하겠습니다. 이후 반복 문의 업무 중 명확한 판단 기준이 있는 영역부터 에이전트화하고, 도구 호출 결과·근거·복구 가능성을 측정할 수 있도록 하겠습니다.

면접에서 설명할 수 있는 주제

- Runnable 체인과 그래프 기반 에이전트의 차이
- LLM의 잘못된 도구 호출을 막는 방법과 제한된 권한 설계
- 체크포인트 보관 범위와 영구 비즈니스 데이터의 분리
- 에이전트 평가 데이터셋의 품질 관리와 재현 가능한 회귀 테스트

자기소개

저는 문제를 기능 단위가 아닌 실제 사용자의 업무 흐름으로 바라봅니다. 에이전트를 단순히 연결하는 것보다 실패를 탐지하고 안전하게 복구하는 구조에 관심이 많습니다. 신입으로서 빠르게 학습하면서도 근거와 측정값을 남기는 개발자가 되겠습니다.